
BATTLE SIMULATOR

Simulacao de Combate em Turnos com Fila Circular Encadeada

Thierry Nathan - 5 slides | Caua Corumba - 5 sides| Joao Vitor Souza - 4 slides

Universidade Federal de Sergipe (UFS)

Bacharelado em Ciencia da Computacao | Algoritmo e Estrutura de Dados I

Grupo 14

Visao Geral do Projeto

+⋮ Contexto do Problema

Sistemas de batalha por turnos sao fundamentais em jogos RPG. Exigem gerenciamento eficiente de entidades em memoria e manipulacao segura de ponteiros.

🎓 Motivacao

Implementar estruturas de dados do zero, sem usar colecoes nativas do Java (ArrayList, LinkedList). Dominar alocao de memoria, ponteiros e encadeamento de nos.

🎯 Objetivo Principal

Construir um simulador de combates 8-bit usando exclusivamente uma Fila Circular Simplesmente Encadeada com No Cabeca para gerenciar combatentes e ordem de turnos com complexidade $O(1)$.

IDEIA CENTRAL DO PROJETO

Herói + Horda de Inimigos

CircularQueue (Fila)

Rotacao de Turnos $O(1)$

Combate em Tempo Real

Toda a comunicacao entre Model e View ocorre via Payloads imutaveis (Java Records), garantindo o desacoplamento total da arquitetura MVC.

Funcionalidades do Sistema

Combate por Turnos

Heroi controlado pelo jogador contra hordas de 2 a 5 inimigos gerados proceduralmente com atributos escalaveis.

Escalonamento de Dificuldade

Inimigos ficam mais fortes a cada round com HP e ataque incrementados, criando progressao de desafio.

Interface Grafica JavaFX

GUI reativa e desacoplada com estilo retro-gaming 8-bit, tipografia monoespacada e CSS customizado.

Log de Batalha em Tempo Real

Registro detalhado de todas as acoes: ataques, danos, curas e mudancas de turno atualizado instantaneamente.

Progressao de Rounds

Derrotada a horda, o heroi sobe de nivel, recupera HP e enfrenta nova horda mais poderosa.

Layout Dinamico Responsivo

Botoes de alvo se redimensionam automaticamente conforme o numero de inimigos sobreviventes (2 a 5).

 Diferencial tecnico: Toda a logica de entidades e turnos e gerenciada pela CircularQueue proprietaria — sem ArrayList, sem LinkedList, sem colecoes nativas do Java.

Fluxo de Funcionamento

1. Inicializacao	2. Gera Horda	3. Loop de Turnos O(1)	4. Processa Dano	5. Verifica Fim
Inicializacao e Geracao Etapa 1: BattleApplication carrega FXML/CSS e instancia MVC. Etapa 2: BattleEngine cria heroi e 2-5 inimigos (procedural). Todos sao enfileirados na CircularQueue.		Loop de Turnos (O(1)) Etapa 3: <code>rotateTurn()</code> remove atacante do inicio e insere no fim da fila — tempo constante. Etapa 4: Se jogador, exhibe botoes de alvo. Se inimigo, IA executa contra-ataque.		Verificacao e Progressao Etapa 5: Se inimigo morto, <code>removeNode()</code> ajusta ponteiros. Se horda vazia, proximo round. Etapa 6: Se heroi morre, Game Over com opcao de reinicio.

ESTADO DA FILA CIRCULAR DURANTE O COMBATE

Estado Inicial (Round 1):
[HEAD] -> Heroi -> Goblin1 -> Goblin2 -> Goblin3 -> [HEAD]

Turno 1 — Heroi ataca:
`rotateTurn()`: Heroi vai para o fim
[HEAD] -> Goblin1 -> Goblin2 -> Goblin3 -> Heroi -> [HEAD]

Goblin2 morre:
`removeNode(Goblin2):`
[HEAD] -> Goblin1 -> Goblin3 -> Heroi -> [HEAD]

Vantagem: sem realocacao de array, sem shift de indices.

Horda derrotada:
Fila esvaziada -> Novo round
[HEAD] -> [HEAD] (vazia)

Nova Horda (Round 2):
[HEAD] -> Heroi -> Goblin1..4 -> [HEAD]

CircularQueue – Fila Circular Encadeada

Tipo: Fila Circular SimplesmenteEncadeada com NoCabeca (Sentinela)
Onde foi utilizada: Gerenciamento da ordem de turnos e armazenamento dos combatentes vivos.
Por que foi escolhida: permite a rotação com baixa complexidade sem dificultar a remoção em qualquer posição

Operacao	Complexidad e	Descricao
rotateTurn()	O(1)	Remove inicio, insere fim
enqueue()	O(1)	Insere no final da fila
removeNode()	O(n)	Busca linear + ajuste ponteiros

Conceito chave: O no cabeca mitiga erros classicos de manipulacao de ponteiros ao eliminar a necessidade de tratar ponteiros nulos em filas vazias – padrao defensivo de engenharia de software.



Vantagens sobre ArrayList

- 1. Sem bugsdeindexacaoquandoelementos sao removidos
- 2. Sem realocacao de arrays ou operacoes de shift
- 3. Insercao e remocao eficientes em qualquer ponto
- 4. Controle total de alocao de memoria e ponteiros
- 5. Rotacao de turnos em tempo constante garantido

MVC ESTRITO

O motor logico nao conhece botoes, telas ou barras de vida. Toda comunicacao ocorre via Payloads imutaveis (Java Records).

VIEW — JavaFX (FXML + CSS) | BattleController

Payloads imutaveis

CONTROLLER — Mediator | Java Records (DTOs)

MODEL — BattleEngine | Domain | Payloads

FACTORY — BattleMenuFactory (UI Dinamica)

Componentes Principais



CircularQueue — Estrutura de dados central. Fila circular encadeada com no cabeca que gerencia todos os combatentes vivos.



BattleEngine— Motor logico do jogo. Contem a fila de combatentes, round atual e regras de batalha.



Domain— Representam personagens (heroi e inimigos). Atributos: nome, HP, HP maximo, ataque, defesa.



BattleController — Vincula eventos da interface ao Model. Traduz acoes do usuario em chamadas de negocio.



BattleMenuFactory — Fabrica de componentes visuais. Calcula layout responsivo dos botoes de alvo.

Principio chave: O Model nunca importa classes da View. A separacao e garantida por DTOs imutaveis, impedindo que a UI altere o estado interno das entidades.

Classes Principais

Classe	Responsabilidade	Atributos Principais	Metodos Principais
CircularQueue	Estrutura de dados central. Gerencia combatentes vivos e ordem de turnos via fila circular encadeada.	head (no cabeca), tail, size	enqueue(), dequeue(), rotateTurn(), remove()
BattleEngine	Motor logico do jogo. Contem regras de batalha, gerencia rounds e estado do combate.	turnQueue, enemyFactory, playerFactory, enemiesList, playerList, currentRound	prepareNextRound(), executeAttack(), findCombatantById(), toRecord()
Combatant	Representa um personagem no jogo (heroi ou inimigo). Encapsula atributos e comportamentos.	name, currentHealth, maxHealth, attackDamage, isPlayer	isAlive(), takeDamage()
BattleController	Mediador MVC. Vincula eventos da interface ao Model e atualiza a View com Payloads.	battleEngine, battleLogView	updateAllUI(), executePlayerAttack(), resetControls(), updateTurnOrder(),
BattleMenuFactory	Fabrica de componentes visuais. Calcula layout responsivo dos botoes de alvo dinamicamente.	container, gridCalculator	createTargetGrid()

Observacao: A classe CircularQueue e generica e pode armazenar qualquer tipo de objeto. No projeto, ela armazena instancias de Entity. O BattleModel nao conhece a existencia da interface grafica – toda comunicacao flui pelo BattleController.

Processamento de Turnos Autônomos

```
private void executePlayerAttack(CombatantRecord target) { 1 usage 8 Cauã Corumba +1
    BattleStatusRecord status = battleEngine.executeAttack(target.id());

    battleLogView.getItems().add(status.actionLog());

    if (status.isGameOver() && status.isVictory()) {
        battleLogView.getItems().add("Vitória! A horda foi derrotada! Avançando de round...");
        startNextRound();
        updateAllUI();
        resetControls();
    }
    else {
        updateAllUI();
        resetControls();
    }
    battleLogView.scrollTo(i: battleLogView.getItems().size() - 1);
}
```

executePlayerAttack

- Funcionamento: Localiza o inimigo real na enemiesList (uma CircularQueue) usando o identificador do CombatantRecord recebido. Aplica o dano e, caso os pontos de vida do alvo zerem, dispara o método .remove() tanto na horda quanto na fila circular de turnos (turnQueue).
- Importância: Garante a consistência estrutural imediata do sistema. Expulsar o nó da memória impede que um inimigo derrotado continue a ser listado como alvo ou que ele receba tempo de CPU no ciclo de turnos

```
private void executeEnemyAttack(){ 1 usage 8 Cauã Corumba +2
    BattleStatusRecord status = battleEngine.executeEnemyTurn();
    String killedName = null;

    battleLogView.getItems().add(status.actionLog());

    if (status.isGameOver() && status.killedTarget() != null) {
        killedName = status.killedTarget().name();
        battleLogView.getItems().add("Derrota... " + killedName + " tombou em combate");
        updateAllUI();

        controlsContainer.getChildren().clear();
        Button restartButton = new Button(s: "Jogar Novamente");
        restartButton.getStyleClass().add("action-button");
        restartButton.setOnAction(ActionEvent e -> handleGameRestart());
        controlsContainer.getChildren().add(restartButton);
    }
    else {
        updateAllUI();
        resetControls();
    }
    battleLogView.scrollTo(i: battleLogView.getItems().size() - 1);
}
```

executeEnemyAttack

- Funcionamento: É invocado de forma automática quando o topo da fila de turnos aponta para um monstro. O algoritmo seleciona o jogador ativo na playersList, extrai o valor de dano do atacante e aplica a mutação de decréscimo de vida via takeDamage(), gerando um log textual.
- Importância: Materializa a automação da Inteligência Artificial (IA) dos inimigos de forma isolada. O motor lógico processa e aplica o ataque de forma autônoma e encapsulada, sem qualquer necessidade de interação ou clique do usuário.


```
private void updateEnemySprites() { 1 usage  👤 João Vitor
    enemySpritesContainer.getChildren().clear();

    List<CombatantRecord> enemies = battleEngine.getEnemiesRecords();

    for (CombatantRecord enemy : enemies) {
        if (!enemy.isDead()) {
            javafx.scene.layout.Region sprite = new javafx.scene.layout.Region();
            sprite.getStyleClass().add("enemy-sprite");
            enemySpritesContainer.getChildren().add(sprite);
        }
    }
}
```

updateEnemySprites

- Funcionamento: Limpa por completo o container visual de renderização gráfica dos oponentes. Em seguida, percorre a lista de snapshots obtida da enemiesList e, para cada criatura que retorne falso no predicado .isDead(), instancia um componente Region acoplando a classe CSS .enemy-sprite.
- Importância: Sincroniza em tempo real a existência física dos monstros em tela. Permite que o jogo oculte e remova visualmente os inimigos abatidos no exato momento da sua eliminação lógico-estrutural.

Sincronização e Renderização Reativa de Componentes Dinâmicos

```
public void updateTurnOrder() { 1 usage  @ João Vitor +1
    turnOrderContainer.getChildren().clear();
    List<CombatantRecord> turnOrder = battleEngine.getTurnOrderRecords();
    List<Label> carouselLabels = TurnOrderFactory.createCarouselNodes(turnOrder);
    turnOrderContainer.getChildren().addAll(carouselLabels);
}
```

updateTurnOrder

- Funcionamento: Limpa completamente o container visual do carrossel na interface. Em seguida, consome a lista imutável de records gerada a partir da rotação atual da CircularQueue e delega para a TurnOrderFactory a criação e estilização dos novos rótulos (Labels).
- Importância: Entrega feedback visual em tempo real para o usuário. Isola as regras de estilo e design (como destacar o primeiro nó ativo com o prefixo ->) fora do controlador principal, mantendo a View reativa.

updateEnemyLifeBars

- Funcionamento: Esvazia o container visual de status dos vilões e realiza uma varredura (foreach) sobre a horda ativa de monstros. Para cada inimigo vivo, invoca a fábrica estática de componentes para desenhar novos indicadores de HP alinhados à esquerda.
- Importância: Sincroniza em tempo real o impacto dos golpes desferidos pelo usuário. Permite que a interface gráfica oculte ou altere a opacidade de monstros derrotados de forma totalmente dinâmica.

updatePlayerLifeBar

- Funcionamento: Limpao painel de status do herói e requisita o snapshot imutável do protagonista ao BattleEngine. O registro (CombatantRecord) é enviado para a CombatantStatusFactory, que reconstrói a barra de progresso e o texto descritivo de HP com alinhamento à direita.
- Importância: Garante a reatividade da interface para as ações sofridas pelo jogador. Isola a lógica estética de posicionamento do painel aliado, mantendo o controlador focado puramente no fluxo de dados

```
private void updateEnemyLifeBars() { 1 usage  @ Thierry Costa +2
    enemyStatusContainer.getChildren().clear();
    List<CombatantRecord> enemies = battleEngine.getEnemiesRecords();
    for (CombatantRecord enemy : enemies) {
        VBox enemyBox = CombatantStatusFactory.createStatusNode(enemy, Pos.TOP_LEFT, textFirst: false);
        enemyStatusContainer.getChildren().add(enemyBox);
    }
}

private void updatePlayerLifeBar() { 1 usage  @ Thierry Costa +2
    playerStatusContainer.getChildren().clear();
    List<CombatantRecord> players = battleEngine.getPlayersRecords();
    for (CombatantRecord player : players){
        VBox playerBox = CombatantStatusFactory.createStatusNode(player, Pos.BOTTOM_RIGHT, textFirst: true);
        playerStatusContainer.getChildren().add(playerBox);
    }
}
```

Rotacao de Turnos

Remove o atacante do inicio da fila e reinsere no final. A natureza circular garante $O(1)$ sem reinicializacao de arrays.

Calculo de Dano

Formula: $\text{dano} = \text{ataqueBase} * \text{fatorRound} - \text{defesaAlvo}$. O fator de round aumenta a dificuldade progressivamente a cada horda derrotada.

Verificacao de Fim de Jogo

Duas verificacoes: (1) se `heroi.isAlive() == false` → Game Over; (2) se fila so contem heroi → vitoria do round, gera nova horda.

Busca para Remocao

Percorre o encadeamento comparando referencias ate encontrar o alvo. Usa dois ponteiros (prev/curr). Complexidade: $O(n)$ no pior caso.

Geracao Procedural de Horda

Gera numero aleatorio de inimigos (2-5) com atributos escalados pelo round atual. Cada novo round = horda maior e mais forte.

Ordenacao Implicita

A fila circular define a ordem de turnos. Nao ha necessidade de ordenacao explicita — a ordem de insercao determina a sequencia.

Decisao de projeto: A escolha da Fila Circular como estrutura unificada eliminou a necessidade de multiplas estruturas auxiliares. Uma unica CircularQueue gerencia tanto a ordem de turnos quanto o conjunto de combatentes vivos, simplificando drasticamente a logica do sistema.

Instanciação Dinâmica e Balanceamento

```
public Enemy[] generateHorde(int currentRound) { 1 usage 3 Cauã Corumba
    int enemiesQty = (int) (Math.random() * (maxEnemys - minEnemys + 1) + minEnemys);

    int multiplier = currentRound-1;
    int currentMinMaxHealth = baseMinHealth + (multiplier * healthGrowth);
    int currentmaxHealth = baseMaxHealth + (multiplier * healthGrowth);
    int currentminAttackDamage = baseMinAttack + (multiplier * attackGrowth);
    int currentmaxAttackDamage = baseMaxAttack + (multiplier * attackGrowth);

    Enemy[] enemiesList = new Enemy[enemiesQty];
    for (int i = 0; i < enemiesQty; i++) {

        int maxHealth = (int) (Math.random() * (currentmaxHealth - currentMinMaxHealth + 1) + currentMinMaxHealth);
        int attackDamage = (int) (Math.random() * (currentmaxAttackDamage - currentminAttackDamage + 1) + currentminAttackDamage);

        Enemy enemy = new Enemy(name: "Inimigo" + (i + 1), maxHealth, attackDamage);
        enemiesList[i] = enemy;
    }
    return enemiesList;
}
```

generateHorde

- Funcionamento Procedural: O método calcula dinamicamente o tamanho da horda (sorteado entre os limites minEnemys e maxEnemys) e utiliza o round atual (currentRound) como um multiplicador matemático para escalar o piso e o teto da vida (maxHealth) e do dano (attackDamage) dos inimigos.
- Instanciação em Lote: Um laço de repetição preenche um vetor nativo (Enemy[]), batizando cada instância de forma sequencial (ex: "Inimigo 1", "Inimigo 2").
- Importância Arquitetural: Isola totalmente a inteligência matemática de balanceamento do jogo fora do motor principal (BattleEngine). Retornar um array limpo impede o acoplamento precoce com a estrutura de ponteiros da fila circular.

```
public Player[] generatePlayer(int currentRound){ 1 usage 3 Cauã Corumba
    int playersQuantity = 1;

    int multiplier = currentRound-1;
    int currentMaxHealth = baseHealth + (multiplier * healthGrowth);
    int currentAttackDamage = baseAttack + (multiplier * attackGrowth);

    Player[] players = new Player[playersQuantity];
    for(int i = 0; i < playersQuantity; i++){

        int maxHealth = currentMaxHealth;
        int attackDamage = currentAttackDamage;

        Player player = new Player(name: "Hero" + (i+1), maxHealth, attackDamage);
        players[i] = player;
    }
    return players;
}
```

generatePlayer

- Crescimento Linear Ativo: Aplica fórmulas baseadas em constantes de progressão (healthGrowth e attackGrowth) multiplicadas pelo avanço das rodadas (currentRound - 1) para atualizar os atributos vitais do herói.
- Consistência de Assinatura: Mantém o mesmo padrão de design da fábrica de inimigos, encapsulando o protagonista criado dentro de um array de tamanho fixo (Player[]).
- Importância Arquitetural: Garante o princípio de Responsabilidade Única (SRP). O motor de jogo não precisa saber como o jogador evolui ou como seus atributos aumentam; ele apenas recebe o payload pronto para inserção ordenada na fila de turnos

Principais Desafios

Imutabilidade de Dados no Core

- Problema: Expor as instâncias reais dos objetos Combatant (Model) para a interface gráfica (View) permitiria modificações indevidas de atributos e vazamento de encapsulamento fora do controle do motor lúdico.
- Solução: Implementação de Java Records como DTOs imutáveis. O motor gera snapshots de dados de leitura que alimentam a View, blindando os ponteiros da Fila Circular de alterações externas.

Desacoplamento Arquitetural (MVC Estrito)

- Problema: Evitar que o controlador JavaFX gerenciasse diretamente regras de negócio complexas de combate ou que o BattleEngine ficasse acoplado a elementos visuais de cena.
- Solução: Centralização de toda a lógica relacional e de transição de estados no Model. O Controller atua puramente como um despachante reativo que escuta eventos e redesenha os containers visuais.

Layout Responsivo e Escalonamento de Alvos

- Problema: Interfaces estáticas e coordenadas fixas quebravam o layout ou geravam lacunas visuais ao instanciar hordas de tamanhos procedurais variáveis (de 2 a 5 inimigos).
- Solução: Geração programática de componentes visuais usando restrições de porcentagem (PercentWidth) dinâmicas em GridPane, recalculando as larguras das células em tempo de execução.

Gerenciamento Procedural de Estados Visuais

- Problema: Controlar manualmente a ativação, desativação e visibilidade de dezenas de componentes visuais (botões de ataque, carrossel de turnos, labels) a cada transição de turno gerava estados inconsistentes e falhas de interface.
- Solução: Centralização do ciclo de atualização numa rotina unificada (resetControls e resetAllUI). A interface é completamente limpa e reconstruída do zero a cada alteração do modelo, garantindo determinismo visual absoluto

Abordagem Fail-Fast

- Todos os metodos de mutacao estrutural aplicam validacoes sob o principio Fail-Fast. Parametros nulos injetados na fila disparam rejeicao instantanea antes da alteracao dos ponteiros de memoria, prevenindo corrupcao de estado e NullPointerException em runtime.

Resultados Obtidos



Sistema Funcional Completo

Simulador de batalha por turnos com interface retro-gaming 8-bit, log em tempo real e progressao com Game Over.



Estrutura Proprietaria

Implementacao funcional de Fila Circular Encadeada com No Cabeca, substituindo .



Arquitetura MVC Estrita

Desacoplamento total entre Model e View com comunicacao exclusiva via DTOs imutaveis (Java Records), garantindo robustez.



Complexidade Otimizada

Rotacao de turnos em $O(1)$. Sem realocacao de arrays, sem shift de indices, sem busca linear para troca de turnos.



Interface Responsiva

Layout dinamico adaptavel a 2-5 inimigos com redimensionamento proporcional dos botoes de alvo.



Codigo Defensivo

Validacoes Fail-Fast em todos os metodos de mutacao estrutural. Prevencao de NullPointerException e corrupcao de ponteiros.

Objetivos atendidos: Dominio de alocao de memoria, ponteiros, estruturas encadeadas, filas circulares, complexidade algoritmica e engenharia de software orientada a objetos com padrao MVC. O projeto cumpre todos os requisitos pedagogicos da disciplina.

Conclusao

✓✓ Aplicacao Pratica de Estruturas de Dados

O BattleSimulator demonstra que Estruturas de Dados tem aplicação direta em sistemas interativos. A Fila Circular Encadeada com Nó Cabeça provou-se adequada para gerenciamento de turnos com eficiência algoritmica.

🔌 Consolidacao do Conhecimento

A transicao de ArrayList para implementação própria consolidou o entendimento sobre ponteiros, alocação dinâmica e encadeamento de nós. Os desafios reforçaram a importancia de padrões defensivos como Fail-Fast.

📦 Engenharia de Software

A arquitetura MVC estrita e os padrões de engenharia aplicados garantiram robustez e manutenibilidade ao sistema. O uso de Java Records como DTOs imutaveis demonstrou como decisoes arquitetonicas impactam diretamente a qualidade do codigo.

💡 Aprendizado Central

Estruturas dedados não são apenas teoricas — suas escolhas impactam diretamente a performance, a robustez e a qualidade do software. A diferenca entre uma ArrayList e uma Fila Circular Encadeada pode determinar se um sistema tem bugs de indexacao ou opera em tempo constante.

OBRIGADO PELA ATENCAO

Battle Simulator — Grupo 14

Universidade Federal de Sergipe (UFS)

Algoritmo e Estrutura de Dados I